WELT 0 · DOPPELSTUNDE 2 · LEVEL 0.2

# W0·2 – Die Werkstatt: Datentypen, Operatoren, Verzweigungen, Schleifen

Datei: `AQ_W0_Akademie_Geruest.txt` (Klasse Werkstatt) · IDE-Level 0.2 · 🎯 Im Lab: **Level 0.2** · Notations-Referenz: AB 1 „Portal“, Teil B

Hinter dem Portal liegt die **Akademie der Baumeister**. Bevor euer Studio Helden schmieden darf, muss jedes Team seine Ausbildung bestehen – und die beginnt in der **Werkstatt**. Hier wird gerechnet, gewogen und gezählt: Wie viel Gold bleibt nach dem Einkauf übrig? Wie viele Kisten braucht der Trankhändler? Welchen Rang verdient ein Lehrling?

Die Werkzeuge dafür kennst du aus der Einführungsphase: Datentypen, Operatoren, Verzweigungen und Schleifen. Heute schärfst du sie – und zwar so, wie das Abitur sie verlangt: **genau**. Denn in Java ist `7 / 2` nicht 3.5, und `-7 % 2` ist nicht 1.

## Infoteil 1: Datentypen und Variablen

Jede Variable hat einen **Datentyp**. Er legt fest, welche Werte sie speichern kann. Die deutschen Typnamen brauchst du für Signaturen im Abitur (Übersicht: AB 1, Teil B, Infoteil 1).

| Java | Abi-Typname | Beispielwerte | Achtung |
|---|---|---|---|
| `int` | Ganzzahl | `0`, `-17`, `2026` | `/` rechnet mit zwei Ganzzahlen **ganzzahlig** |
| `double` | Kommazahl | `3.5`, `-0.25`, `7.0` | Dezimal**punkt**, kein Komma |
| `boolean` | Wahrheitswert | `true`, `false` | im Struktogramm: wahr / falsch |
| `char` | Zeichen | `'a'`, `'Z'`, `'7'`, `' '` | **einfache** Anführungszeichen, genau ein Zeichen |
| `String` | Zeichenkette | `"Drache"`, `""` | **doppelte** Anführungszeichen; Vergleich mit `equals` |

- **Deklaration und Zuweisung:** `int gold;` legt die Variable an, `gold = 50;` weist einen Wert zu, `int gold = 50;` macht beides. Im Struktogramm schreibst du die Zuweisung mit Pfeil: `gold ← 50`.
- **= weist zu, == vergleicht.** Im Struktogramm steht für den Vergleich ein einfaches `=`.
- **Automatische Umwandlung:** Eine Ganzzahl passt in eine Kommazahl-Variable (`double d = 7;` speichert 7.0). Umgekehrt braucht man einen **Cast** `(int)` – und der schneidet die Nachkommastellen ab.

## Aufgabe 0: Blitz-Warm-up: Was rechnet Java? PAPIER ★

Notiere zu jedem Ausdruck den Wert. Erst vermuten – dann mit Infoteil 2 prüfen und mit Farbe korrigieren. Kommazahlen schreibst du mit Punkt (z. B. `2.0`), Wahrheitswerte als `true` / `false`.

| Ausdruck | `23 / 4` | `23 % 4` | `4 % 23` | `-9 / 2` | `-9 % 2` | `10 - 3 * 2 % 4` |
|---|---|---|---|---|---|---|
| Wert | | | | | | |

| Ausdruck | `(double) 9 / 2` | `(double) (9 / 2)` | `(int) 7.9` | `1 / 2 * 4.0` | `7 > 3 && 2 > 5` | `!(3 == 3) \|\| 4 != 4` |
|---|---|---|---|---|---|---|
| Wert | | | | | | |

## Infoteil 2: Operatoren

| Operator | Bedeutung | Beispiel | Ergebnis |
|---|---|---|---|
| `+ - *` | wie in der Mathematik, Punkt vor Strich | `3 + 4 * 2` | 11 |
| `/` mit zwei Ganzzahlen | **ganzzahlige Division** – der Rest fällt weg | `17 / 5` | 3 |
| `/` mit einer Kommazahl | normale Division | `17 / 5.0` | 3.4 |
| `%` (mod) | **Rest** der ganzzahligen Division; `*`, `/` und `%` haben gleichen Rang | `17 % 5` | 2 |
| `%` mit negativer Zahl | Der Rest hat das Vorzeichen der **linken** Zahl | `-7 % 2` | −1 (nicht 1!) |
| `(int) (double)` | Typumwandlung (Cast); `(int)` schneidet ab, rundet nicht | `(int) 3.99` | 3 |

- **Vergleiche** `==  !=  <  <=  >  >=` liefern einen Wahrheitswert. Im Struktogramm: `=  ≠  <  ≤  >  ≥`.
- **Logische Operatoren:** && (und), `||` (oder), `!` (nicht). Java wertet von links aus und hört auf, sobald das Ergebnis feststeht: In `i < r.length && r[i] > 0` wird `r[i]` nur gelesen, wenn der Index gültig ist.
- **Typische Anwendungen von `%`:** `n % 2 == 0` (gerade?), `n % 10` (letzte Ziffer), `n / 10` (letzte Ziffer streichen), `a % b == 0` (b teilt a).

## Achtung Abi: Die Casting-Falle

| Ausdruck (summe = 50) | Was passiert? | Ergebnis |
|---|---|---|
| `(double) summe / 3` | erst umwandeln, dann teilen → Kommazahl-Division | 16.666… ✓ |
| `summe / 3.0` | eine Kommazahl beteiligt → Kommazahl-Division | 16.666… ✓ |
| `(double) (summe / 3)` | erst **ganzzahlig** teilen (16), dann umwandeln | 16.0 × |

## Infoteil 3: Zufall und Runden (EH 2025)

`Math.random()` liefert eine Kommazahl z mit **0 ≤ z < 1** – die 1 selbst kommt nie vor. Daraus baust du eine **ganzzahlige Zufallszahl im Bereich** Schritt für Schritt, hier für einen Würfel (1 bis 6):

| Ausdruck | mögliche Werte |
|---|---|
| `Math.random()` | 0 ≤ z < 1 (Kommazahl) |
| `Math.random() * 6` | 0 ≤ … < 6 (Kommazahl) |
| `(int) (Math.random() * 6)` | 0, 1, 2, 3, 4, 5 (Cast schneidet ab) |
| `(int) (Math.random() * 6) + 1` | 1, 2, 3, 4, 5, 6 |

**Allgemein** für eine Zahl von `von` bis `bis` (beide möglich, alle gleich wahrscheinlich): `(int) (Math.random() * (bis - von + 1)) + von` – denn `bis - von + 1` ist die **Anzahl** der möglichen Werte.

**Ganzzahlig runden** (kaufmännisch, für x ≥ 0): `(int) (x + 0.5)`. Beispiel: 2.4 + 0.5 = 2.9 → 2 und 2.5 + 0.5 = 3.0 → 3. In der Akademie sind `Math.round`, `Math.floor`, `Math.ceil` und `Math.pow` tabu – du baust Runden und Potenzieren selbst.

**Achtung Abi: Klammern beim Cast**

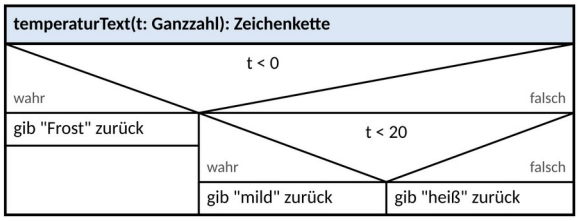
`(int) Math.random() * 6 + 1` liefert **immer 1**: Der Cast wirkt nur auf `Math.random()` und macht daraus sofort 0. Die Klammer muss um das ganze Produkt: `(int) (Math.random() * 6) + 1`.

## Infoteil 4: Verzweigungen und Schleifen – Ergänzungen zu AB 1

Die Grundelemente (Verzweigung, Solange-, Zähl- und Für-jedes-Schleife) stehen auf AB 1, Teil B, Infoteil 2. Neu sind hier die **Verzweigungskette** und die **fußgesteuerte Schleife**.

**else-if-Kette:** Die Bedingungen werden **der Reihe nach** geprüft; nur der erste zutreffende Zweig wird ausgeführt. Im Struktogramm wird daraus eine **geschachtelte** Verzweigung im Falsch-Zweig.

```
public static String temperaturText(int t) {
    if (t < 0) {
        return "Frost";
    } else if (t < 20) {
        return "mild";
    } else {
        return "heiß";
    }
}
```




| Schleife | Java | Struktogramm | Wann? |
|---|---|---|---|
| kopfgesteuert | `while (bed) { … }` | solange bed | Anzahl der Durchläufe unbekannt; evtl. 0-mal |
| Zählschleife | `for (int i = 0; i < n; i++) { … }` | für i von 0 bis n − 1 | Anzahl bekannt; „bis“ ist **einschließlich** |
| fußgesteuert | `do { … } while (bed);` | – (im Abitur kaum verwendet) | Rumpf läuft **mindestens einmal**; lässt sich immer mit while schreiben |

## Aufgabe 1: Der Typ-Detektiv

PAPIER ★

Ein Lehrling hat zwölf Anweisungen in die Werkstatt-Datei getippt. Nicht alle nimmt der Compiler an.

**a)** Kreuze an, ob die Anweisung kompiliert. **[3 BE]**

**b)** Gib bei kompilierenden Anweisungen den gespeicherten Wert an, sonst eine kurze Begründung. **[4 BE]**

| Nr. | Anweisung | kompiliert? | Wert bzw. Grund |
|---|---|---|---|
| 1 | `int a = 7.0;` | ☐ ja ☐ nein | |
| 2 | `double d = 7;` | ☐ ja ☐ nein | |
| 3 | `char c = "x";` | ☐ ja ☐ nein | |
| 4 | `String s = 'x';` | ☐ ja ☐ nein | |
| 5 | `boolean b = 0;` | ☐ ja ☐ nein | |
| 6 | `int n = (int) 7.9;` | ☐ ja ☐ nein | |
| 7 | `double q = 7 / 2;` | ☐ ja ☐ nein | |

| Nr. | Anweisung | kompiliert? | Wert bzw. Grund |
|---|---|---|---|
| 8 | `int m = 10 / 4.0;` | ☐ ja ☐ nein | |
| 9 | `String t = "Gold: " + 5 + 3;` | ☐ ja ☐ nein | |
| 10 | `String u = 5 + 3 + " Gold";` | ☐ ja ☐ nein | |
| 11 | `int k = (int) 'A' + 2;` | ☐ ja ☐ nein | |
| 12 | `boolean g = 3 < 5 < 7;` | ☐ ja ☐ nein | |

**Tipp:** Bei 9 und 10 arbeitet `+` von links nach rechts: Sobald eine Zeichenkette beteiligt ist, wird **verkettet** statt addiert. Bei 11 hilft die ASCII-Tabelle: `'A'` hat den Wert 65.

## Aufgabe 2: Wechselgeld in der Werkstatt

PAPIER ★★

In der Akademie gilt: 1 Gold = 100 Kupfer, 1 Silber = 10 Kupfer. Der Händler zahlt einen Betrag in möglichst wenigen Münzen aus.

```
1  int betrag = 487;
2  int gold = betrag / 100;
3  betrag = betrag % 100;
4  int silber = betrag / 10;
5  int kupfer = betrag % 10;
```

**a)** Erstelle eine Tracetabelle: eine Zeile pro Anweisung, eingetragen wird nur der neue Wert. **[2 BE]**

| Zeile | betrag | gold | silber | kupfer |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |

**b)** Ein Lehrling vertauscht die Zeilen 3 und 4. Gib an, welcher Wert sich ändert, und nenne den neuen Wert. **[2 BE]**

**c)** Der Händler berechnet den Durchschnittspreis dreier Tränke und rundet ihn. Gib die Werte der vier Variablen an. **[3 BE]**

```
int summe = 50;
double schnitt = summe / 3;
double schnitt2 = (double) summe / 3;
int gerundet = (int) (schnitt2 + 0.5);
int abgeschnitten = (int) schnitt2;
```

| Variable | Wert |
|---|---|
| schnitt | |
| schnitt2 | |
| gerundet | |
| abgeschnitten | |

**d)** Gib `gerundet` für `summe = 49` an und begründe mit einer kurzen Rechnung. **[1 BE]**

## Aufgabe 3: Die Würfel der Werkstatt

PAPIER ★★

**a)** Gib für jeden Ausdruck den kleinsten und den größten möglichen Wert an. **[3 BE]**

| Ausdruck | kleinster Wert | größter Wert |
|---|---|---|
| `(int) (Math.random() * 6)` | | |
| `(int) (Math.random() * 6) + 1` | | |
| `(int) Math.random() * 6 + 1` | | |
| `(int) (Math.random() * 11) - 5` | | |
| `(int) (Math.random() * 20) + 10` | | |

**b)** Gib einen Ausdruck an, der eine ganzzahlige Zufallszahl von −3 bis 3 liefert (alle Werte gleich wahrscheinlich). **[2 BE]**

**c)** Der Würfelmeister würfelt mit zwei Würfeln und addiert. Sein Lehrling meint: „Das ist dasselbe wie eine Zufallszahl von 2 bis 12." Beurteile die Aussage. **[2 BE]**

## Aufgabe 4: Rangabzeichen – die Reihenfolge zählt

PAPIER ★★

Die Akademie vergibt Rangabzeichen nach Erfahrungspunkten (xp): unter 100 Novize, ab 100 Lehrling, ab 250 Geselle, ab 500 Meister. Ein Lehrling hat dafür diese Methode geschrieben:

```
1  public static String rang(int xp) {
2      String r = "Novize";
3      if (xp >= 100) {
4          r = "Lehrling";
5      } else if (xp >= 250) {
6          r = "Geselle";
7      } else if (xp >= 500) {
8          r = "Meister";
9      }
10     return r;
11 }
```

**a)** Gib die Rückgabewerte an. **[2 BE]**

| Aufruf | `rang(50)` | `rang(120)` | `rang(300)` | `rang(800)` |
|---|---|---|---|---|
| **Rückgabe** | | | | |

**b)** Erläutere, warum die Ränge Geselle und Meister nie vergeben werden. **[2 BE]**

**c)** Stelle eine korrigierte Fassung der Methode als Struktogramm nach EH § 5 dar (mit Kopfzeile und „gib … zurück“). **[4 BE]**

**d)** Ein anderer Lehrling behält die Reihenfolge bei, ersetzt aber jedes `else if` durch ein einzelnes `if`. Prüfe mit `rang(300)` und `rang(800)`, ob die Methode dann richtig arbeitet, und begründe. **[2 BE]**

## Aufgabe 5: Die Zauberfolge – Schleifen tracen

PAPIER ★★

Ein alter Zauber verändert eine Zahl so lange, bis sie 1 ist: Gerade Zahlen werden halbiert, ungerade verdreifacht und um 1 erhöht. Die Methode zählt die Schritte.

```
1  public static int schritte(int n) {
2      int s = 0;
3      while (n != 1) {
4          if (n % 2 == 0) {
5              n = n / 2;
6          } else {
7              n = 3 * n + 1;
8          }
9          s++;
10     }
11     return s;
12 }
```

**a)** Erstelle eine Tracetabelle für den Aufruf `schritte(6)`: erste Zeile vor der Schleife, dann eine Zeile am Ende jedes Durchlaufs. Gib den Rückgabewert an. **[3 BE]**

| Durchlauf | n % 2 == 0 (w/f) | n | s |
|---|---|---|---|
| vor der Schleife | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

Rückgabewert: _____

**b)** In einer zweiten Fassung wird die Schleife fußgesteuert: `do { … } while (n != 1);` – der Rumpf bleibt gleich. Gib einen Startwert n ≥ 1 an, für den beide Fassungen **verschiedene** Ergebnisse liefern, und nenne beide Ergebnisse. **[2 BE]**

**c)** Gib die Ausgabe dieses Programmausschnitts an. **[2 BE]**

```
for (int z = 1; z <= 3; z++) {
    String zeile = "";
    for (int s = 1; s <= 5; s++) {
        if (s % z == 0) {
            zeile = zeile + "#";
        } else {
            zeile = zeile + ".";
        }
    }
    println(zeile);
}
```

| Ausgabe |
|---|
| |
| |
| |

## Aufgabe 6: Bauplan und Code

PAPIER ★★

Die Akademie verwahrt ihre Baupläne als Struktogramme. Übersetze in beide Richtungen.

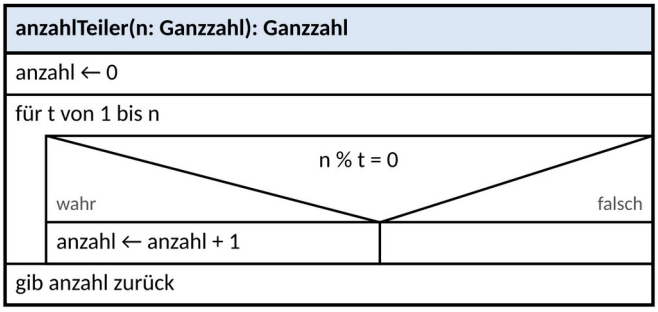



*Abbildung 1: Struktogramm der Methode anzahlTeiler*

**a)** Implementiere die Methode `anzahlTeiler` aus Abbildung 1 in Java. **[3 BE]**

**b)** Gib `anzahlTeiler(12)` und `anzahlTeiler(7)` an. Beschreibe, welche Zahlen n ≥ 2 genau den Rückgabewert 2 haben. **[2 BE]**

Der Schatzmeister legt Gold an: Jedes Jahr kommt ein Zehntel (ganzzahlig) dazu.

```
public static int jahreBisZiel(int gold, int ziel) {
    int jahre = 0;
    while (gold < ziel) {
        gold = gold + gold / 10;
        jahre++;
    }
    return jahre;
}
```

**c)** Stelle die Methode `jahreBisZiel` als Struktogramm nach EH § 5 dar. **[3 BE]**

**d)** Gib `jahreBisZiel(100, 150)` an. Notiere dazu die Werte von gold nach jedem Jahr. **[2 BE]**

**e)** Für manche Startwerte endet die Methode nie (bei ziel > gold). Gib alle diese Startwerte für gold an und begründe. **[2 BE]**

**🎯 Im Lab üben: Level 0.2 „Die Werkstatt: Datentypen und Kontrollstrukturen“ – Punkte gibt es dort automatisch**

Passend zu den Infoteilen 1–4 und den Aufgaben 0–8: Typen und Werte bestimmen, Verzweigungen und Schleifen tracen, Bug-Hunt – jede Antwort wird sofort geprüft. Danach geht es in der Online-IDE mit IDE-Level 0.2 weiter.

## Aufgabe 7: Level 0.2 – Die Werkstatt

IDE ★ – ★★★

**a)** Lege in der Online-IDE den Workspace **AlgoQuest_W0_Akademie** an (oder öffne den ausgeteilten) und darin eine Datei mit dem Inhalt von `AQ_W0_Akademie_Geruest.txt`. Lies die ANLEITUNG am Dateianfang. Heute arbeitest du nur in der Klasse **Werkstatt**.

**b)** Über jeder Methode stehen ihre Signatur in Abi-Notation und eine Beschreibung. Ersetze `// TODO Level 0.2` und das vorläufige `return` durch deine Lösung. Starte im Testrunner die Testklasse `L02WerkstattTest`.

| Stufe | Methoden (Signatur wie im Abitur) | Tests beginnen mit |
|---|---|---|
| ★ Basis | istGerade(n: Ganzzahl): Wahrheitswert<br>restGold(gold: Ganzzahl, preis: Ganzzahl): Ganzzahl<br>kistenBenoetigt(anzahl: Ganzzahl, proKiste: Ganzzahl): Ganzzahl | `basis_` |
| ★★ Erweiterung | schadensstufe(schaden: Ganzzahl): Zeichenkette<br>durchschnitt(a: Ganzzahl, b: Ganzzahl, c: Ganzzahl): Kommazahl<br>quersumme(n: Ganzzahl): Ganzzahl | `erweiterung_` |
| ★★★ Experte | potenz(basis: Ganzzahl, exponent: Ganzzahl): Ganzzahl<br>ggT(a: Ganzzahl, b: Ganzzahl): Ganzzahl<br>istPrimzahl(n: Ganzzahl): Wahrheitswert | `experte_` |

Eine Stufe ist geschafft, wenn **alle** Tests mit diesem Anfang grün sind. Lies bei roten Tests die Meldung – sie nennt den Fall, der noch nicht klappt.

**c)** **Vorhersage ohne Gerät:** Im Hauptprogramm ganz unten steht das „Wert-Rätsel“ mit `int jahr = 2026;`. Notiere, welche Quersumme und welchen Wert von `jahr` das Programm ausgibt. Starte es erst danach und vergleiche.

---

### Tipps aus der Werkstatt

- **istGerade:** Teste auch negative Zahlen. `-3 % 2` ergibt `-1` – frage deshalb nach `== 0`.
- **kistenBenoetigt:** Ob eine angefangene Kiste dazukommt, verrät dir `%`. Ohne `Math.ceil`!
- **durchschnitt:** Die Casting-Falle aus Infoteil 2 wartet hier auf dich.
- **quersumme:** `n % 10` ist die letzte Ziffer, `n / 10` streicht sie.
- **ggT (Euklid):** ggT(a, b) = ggT(b, a % b) und ggT(a, 0) = a. Ersetze (a, b) so lange durch (b, a % b), bis b = 0 ist.
- **istPrimzahl:** Es genügt, Teiler t mit `t * t <= n` zu prüfen.

## Aufgabe 8: Bug-Hunt in der Werkstatt

PAPIER ★★

Vier Lehrlinge haben Werkstatt-Methoden abgegeben. In jeder steckt **genau ein** Fehler.

**Methode A** – soll true liefern, wenn n ungerade ist:

```
1  public static boolean istUngerade(int n) {
2      return n % 2 == 1;
3  }
```

**Methode B** – soll die Summe 1 + 2 + … + n liefern:

```
1  public static int summeBis(int n) {
2      int summe = 0;
3      int i = 1;
4      while (i <= n) {
5          summe = summe + i;
6      }
7      return summe;
8  }
```

**Methode C** – soll „pleite“ liefern, wenn kein Gold mehr da ist, sonst „flüssig“:

```
1  public static String kasse(int gold) {
2      if (gold = 0) {
3          return "pleite";
4      }
5      return "flüssig";
6  }
```

**Methode D** – soll true liefern, wenn jemand Jugendlicher ist (13 bis 17 Jahre):

```
1  public static boolean istJugendlich(int alter) {
2      return alter > 12 || alter < 18;
3  }
```

**a)** Gib für jede Methode die fehlerhafte Zeile an, beschreibe den Fehler und korrigiere ihn. **[4 BE]**

| Methode | Zeile | Fehler | Korrektur |
|---|---|---|---|
| A | | | |
| B | | | |
| C | | | |
| D | | | |

**b)** Welcher der vier Fehler wird schon vom **Compiler** gemeldet? Welcher fällt erst beim **Ausführen** auf – und woran? **[2 BE]**

**c)** Gib zu Methode A und zu Methode D je einen Aufruf an, der ein falsches Ergebnis liefert. **[2 BE]**

### Für Schnelle: Zahlen mit Eigenschaften

Schreibe in einer eigenen Klasse `Zahlenkunde` passende Methoden mit Signatur und teste sie im Hauptprogramm. Nutze dabei Methoden aus deiner Werkstatt (Aufruf z. B. `Werkstatt.potenz(3, 3)`).

- **Perfekte Zahlen:** Eine Zahl ist perfekt, wenn sie gleich der Summe ihrer echten Teiler ist (6 = 1 + 2 + 3). Finde alle perfekten Zahlen unter 10 000. Es sind erstaunlich wenige.
- **Armstrong-Zahlen:** Eine dreistellige Zahl, die gleich der Summe der dritten Potenzen ihrer Ziffern ist: $153 = 1^3 + 5^3 + 3^3$. Finde alle vier.
- **Rekordhalter der Zauberfolge:** Welche Startzahl bis 1000 braucht mit `schritte` aus Aufgabe 5 die meisten Schritte? Probiere auch n = 27.

Side-Quest: freiwillig und ohne Punkte – stellt eure Ergebnisse im Team oder der Lehrkraft vor.